Async / Await

Why it matters

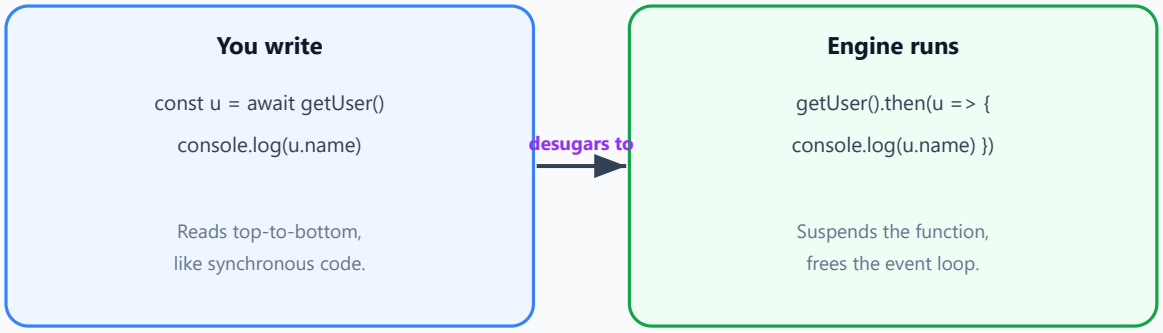
`async/await` is how modern Node code reads: top to bottom, like synchronous logic, even though it's fully asynchronous underneath. It removes the nesting of callbacks and the boilerplate of long `.then` chains, and it lets you use ordinary language features — `try/catch`, `for` loops, `if`, early `return` — with asynchronous values. For most day-to-day backend code, this is the pattern you'll write.

But its readability is also a trap. Because `await` makes async code *look* sequential, it's dangerously easy to make everything sequential by accident — awaiting three independent requests one after another when they could have run together, turning a 100ms operation into 300ms. Interviewers love this exact scenario: give the candidate a snippet with three awaits and ask how to make it faster. Knowing that `async/await` is just promise syntax, and knowing when to break out of the sequential reading to run things in parallel, is what separates people who *use* it from people who *understand* it.

The concept, simply

Marking a function `async` does two things: it makes the function always return a promise (whatever you return becomes the fulfilment value; whatever you throw becomes the rejection reason), and it allows `await` inside it. `await` takes a promise, pauses the async function until that promise settles, and then evaluates to the fulfilled value — or throws the rejection reason as if it were a normal exception.

The key mental correction: `await` **pauses your function, it does not block the thread.** While your function is suspended at an `await`, the event loop is free to run other work — handle other requests, fire timers, process I/O. Nothing is frozen; only *this one function* is parked until its promise is ready.

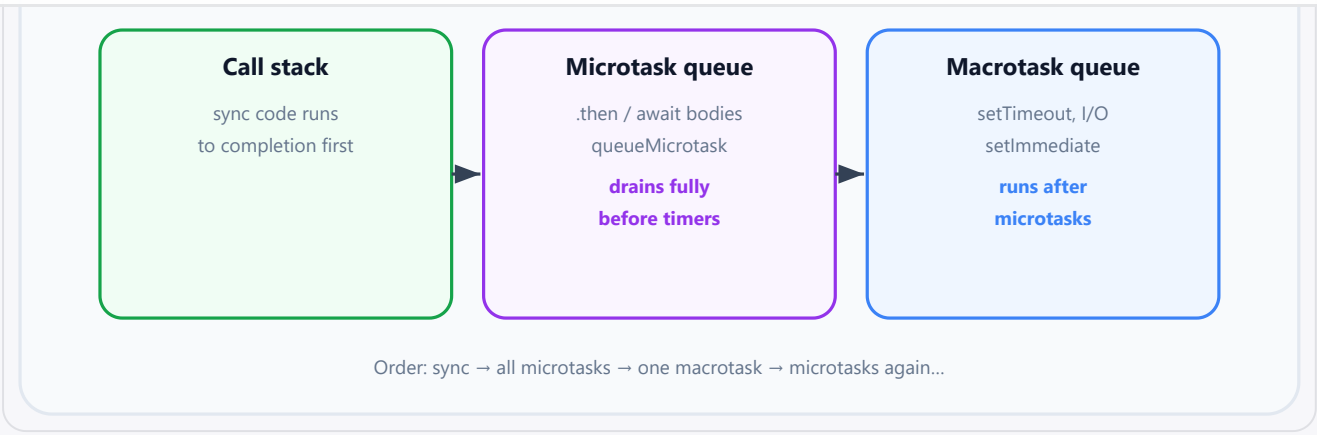


await reads like synchronous code but the engine turns it into a suspended continuation — the thread stays free.

How it works

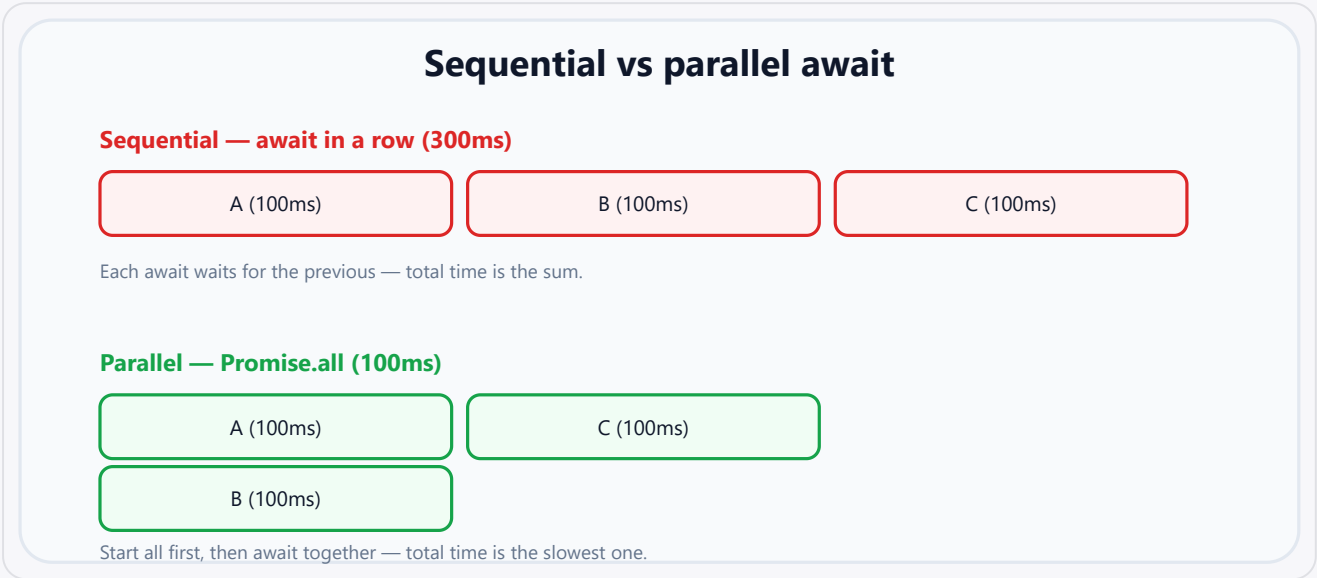
An `async` function is a state machine. When execution reaches an `await`, the function *suspends*: it saves its local state, attaches a continuation to the awaited promise (conceptually a `.then`), and returns control to the caller. The rest of the function body becomes that continuation, scheduled as a microtask when the promise settles. So `const u = await getUser()` is, roughly, `getUser().then(u ⇒ { /* rest of function */ })`. This is why everything after an `await` runs on a later microtask, never synchronously.

Because the continuation runs as a microtask, the same ordering rules from promises apply: code after an `await` runs before a `setTimeout(fn, 0)` scheduled beforehand. And because a rejected awaited promise *throws* at the `await` point, ordinary `try/catch` works again — the async error handling that callbacks couldn't offer is restored, because the language rethrows the rejection synchronously into your paused function.



The code after an await resumes as a microtask, ahead of any timer or I/O callback queued at the same time.

The performance model follows directly. Two `await`s in a row mean the second operation doesn't even *start* until the first has finished — that's sequential, and the total time is the sum. To run independent operations concurrently you must **start them all first** (call the functions, collect the promises) and only then `await` them together, typically with `Promise.all`. Then the total time is roughly the slowest single operation.



Awaiting in a row sums the durations; starting all the promises first and awaiting together takes only as long as the slowest.

Unbounded parallelism is the failure mode on the far side of the sequential trap. `await Promise.all(tenThousandIds.map(fetchRow))` starts all ten thousand operations at once, opening as many sockets or file handles, and typically collapses under its own weight or trips a downstream rate limit. The middle ground is **bounded concurrency**: run at most N operations in flight at a time. A well-worn library such as `p-limit` does

A minimal concurrency limiter: N workers drain a shared queue, and results stay in input order because each worker writes to the index it claimed.

JAVASCRIPT Copy

```
async function mapLimit(items, limit, fn) {
  const results = new Array(items.length);
  let next = 0;
  async function worker() {
    while (next < items.length) {
      const i = next++;              // claim an index — safe on one threa
      results[i] = await fn(items[i], i);
    }
  }
  const size = Math.min(limit, items.length);
  await Promise.all(Array.from({ length: size }, worker));
  return results;
}

// At most 5 requests in flight at once, regardless of ids.length.
const rows = await mapLimit(ids, 5, fetchRow);
```

One more subtlety about `await Promise.all`: it rejects with the *first* rejection the instant it arrives, but the other in-flight promises are **not** cancelled — they run to completion and their results are simply discarded. If one of those stragglers later rejects with nothing awaiting it, you can still get an `unhandledRejection` after your `catch` has already run. When partial success is acceptable, prefer `Promise.allSettled` so every outcome is captured and nothing is left dangling.

Code examples

The sequential trap and its fix. The first version takes the sum of all three; the second takes the max because the requests overlap:

JAVASCRIPT Copy

```
// each await waits for the previous (≈300ms).
async function slow() {
  const a = await load('a'); // 100ms
  const b = await load('b'); // another 100ms
  const c = await load('c'); // another 100ms
  return [a, b, c];
}

// FAST — parallel: start all three, then await together (≈100ms).
async function fast() {
  const [a, b, c] = await Promise.all([load('a'), load('b'), load('c')]);
  return [a, b, c];
}
```

Sequential is correct — and required — when steps *depend* on each other. You cannot parallelise what needs the previous result:

JAVASCRIPT Copy

```
async function checkout(userId) {
  const user = await getUser(userId);        // must run first
  const cart = await getCart(user.cartId);   // needs user
  const charge = await pay(cart.total);      // needs cart
  return charge.receipt;
}
```

Error handling reads like synchronous code — the rejected promise throws at the `await`, and `catch` receives it. `finally` runs either way:

JAVASCRIPT Copy

```
async function safeLoad(id) {
  try {
    const data = await load(id);
    return process(data);
  } catch (err) {
    console.error('load failed:', err.message);
    return null;                  // graceful fallback
  } finally {
    console.log('attempt finished'); // always runs
```

Iterating with `await` in a `for ... of` is sequential by design — use it when order matters or to limit concurrency. For unordered parallelism, map to promises and `Promise.all`. Note that `forEach` does *not* await:

JAVASCRIPT Copy

```
// Sequential, ordered — one at a time.
for (const id of ids) {
  await handle(id);
}

// Parallel — all at once, results in order.
const results = await Promise.all(ids.map((id) => handle(id)));

// BUG: forEach ignores the returned promise — these run "in the backgroun
// and the outer function does NOT wait for them.
ids.forEach(async (id) => { await handle(id); }); // don't do this
```

Common mistakes & gotchas

- **The forgotten `await`.** Dropping `await` gives you the pending *promise object*, not the value — so `if (user.isAdmin)` reads a property off a Promise and is silently truthy/undefined. The operation also isn't sequenced.
- **Accidental sequential awaits.** Awaiting independent operations one per line serialises them needlessly. Start them together and `await Promise.all`.
- **Floating promise from a missing `await` or `return`.** The function returns before the async work finishes, and any rejection becomes unhandled.
- **`await` inside `Array.forEach` / `map` without collecting.** `forEach` ignores the returned promise entirely; an `async` callback runs detached. Use `for ... of` for sequential or `Promise.all(map( ... ))` for parallel.
- **Unbounded parallelism.** `Promise.all(hugeArray.map(fetch))` can open thousands of sockets at once and exhaust resources. Batch or use a concurrency limiter.

- **Top-level `await` availability.** It works in ES modules (`.mjs` or `"type":"module"`) but not in classic CommonJS module scope.

Best practices

- Default to `async/await` for readability; drop to `Promise.all` the moment operations are independent.
- Start independent promises *before* awaiting — collect them, then `await Promise.all`.
- Use `try/catch/finally` for error handling and cleanup; it works naturally with `await`.
- Use `for ... of` with `await` for ordered/sequential work; never rely on `forEach` to await.
- Cap concurrency for large collections with batching or a limiter (e.g. p-limit) instead of a bare `Promise.all` over thousands of items.
- Await or return every promise you create; a lint rule for floating promises catches the rest.

⭐ Interview lens

> **Q: Does `await` block the event loop?**
> No. It suspends only the current async function and returns control to the event loop, which is free to run other work. The thread is never frozen — a common misconception.

> **Q: These three awaits take 300ms — make them 100ms.**
> They're independent but awaited sequentially, so each waits for the last. Start all three (call the functions to get promises) and `await Promise.all([ ... ])`; total time drops to the slowest single call.

Always a promise. A returned value becomes the fulfilment; a thrown error (or a rejected awaited promise) becomes the rejection. Even `return 5` yields `Promise<5>`.

Q: Why doesn't `await` work inside `forEach`? (trap)

`forEach` doesn't look at the promise its callback returns, so it doesn't wait — the loop finishes immediately while the async callbacks run detached. Use `for ... of` for sequential or `Promise.all(arr.map( ... ))` for parallel.

Q: What is `async/await` compiled to conceptually?

A promise-based state machine. Each `await` suspends the function and attaches its continuation to the awaited promise as a microtask — essentially the code after it becomes a `.then` callback.

Q: When does `return await x` differ from `return x`?

Inside a `try` block: `return await` lets the local `catch` handle a rejection, whereas `return x` hands the still-pending promise to the caller and skips your `catch`. Outside a `try` the difference is a negligible extra microtask.

Cheat-card

- `async` **fn** → always returns a promise; `return` = fulfil, `throw` = reject.
- `await` pauses the function, frees the event loop — never blocks the thread.
- **Sequential** awaits sum durations; use only for dependent steps.
- **Parallel:** start promises first, then `await Promise.all([ ... ])`.
- **Errors:** `try/catch` works because a rejected await throws.
- **Loops:** `for ... of` awaits; `forEach` does not. Cap huge parallelism.

QUICK CHECK — no marks, just for you.

Node.js Backend Kit

1. Does `await` block the event loop while waiting for a promise to settle?

- ◯ Yes, it freezes the whole thread
- ◯ Yes, but only for I/O
- ◯ No, but it blocks all other async functions
- ◯ No — it suspends only the current async function and returns control to the event loop

Check answer

2. Three independent 100ms awaits run one per line, taking ~300ms total. How do you make it ~100ms?

- ◯ Start all three (collect the promises), then `await Promise.all([...])` so they overlap
- ◯ Add `await` before each line twice
- ◯ Wrap each in `setTimeout`
- ◯ Use a `for...of` loop with await

Check answer

3. Why does `await` inside `Array.forEach` fail to wait for the async work?

- ◯ forEach converts callbacks to synchronous functions
- ◯ forEach ignores the promise its callback returns, so it does not wait — the async callbacks run detached
- ◯ forEach only works with synchronous arrays

Check answer

Previous
Promises

Next
Error Handling in Async Code

LEARN MORE

ARTICLES & DOCS

- async function (MDN)
  MDN
- Using promises (MDN)
  MDN

VIDEOS



- async/await simply explained
  YouTube



- async/await vs promises
  YouTube



- async/await is easy!
  YouTube

Hand-picked references for this lesson.